

Fiori Elements 開發課程 15 (延伸篇) : Message Class 與多語系訊息維護

環境：BTP ABAP Environment Trial ([ZI_RC05_NOTE](#) / [ZBP_I_RC05_NOTE](#) ，套件 [ZRAPCLOUD](#) ，延續 [fe14](#) 的 [validateTitleContent](#))

Lecture

為什麼要延伸這一段

[fe14](#) 選 [new_message_with_text](#) 是為了讓新增的後端改動維持最小——文字直接寫死在 ABAP 程式碼裡，沒有多語系翻譯的好處，也沒辦法被別的程式重複引用同一則訊息。正式專案常見的需求剛好相反：同一則訊息可能被多處引用、需要支援多國語言，這時候要用真正的 **Message Class** (T100 ，傳統上用 [SE91](#) 維護) 。但這個系統是 **ABAP Cloud (BTP ABAP Environment)** ，沒有 **SAP GUI** 、沒有 **T-code**——[SE91](#) 根本不存在，這一課要解決兩個問題：① Message Class 在這種環境裡到底怎麼建立、怎麼在 RAP Validation 裡引用；② 沒有 [SE63](#) / [SOTR_EDIT](#) 這些傳統多語系維護工具，多語言訊息文字要怎麼維護。

查證：Message Class 在這個環境裡真實存在，只是沒有 GUI 工具

直接在系統上查證 (不是憑印象猜的) ：搜尋這個共用 BTP Trial 系統裡所有 [MSAG](#) (Message Class) 型別物件，找到大量其他學員留下的真實範例——[Z01_MESSAGE_FLIGHT](#) (Flight Connection Messages) 、[ZAS_MC_TRAVEL](#) (Message class for Travel app) 、[ZBS_DEMO_RAP](#) (Demo RAP) 等等，證實這個物件類型在 ABAP Cloud 上是真的能用、正常運作的機制，不是查無此路的死路。讀其中一個的內容確認物件本質：

```
<mc:messageClass adtcore:masterLanguage="EN" adtcore:name="Z01_MESSAGE_FLIGHT"
adtcore:type="MSAG/N" ...>
  <mc:messages mc:msgno="001" mc:msgtext="Flight number &amp;1 and &amp;2 already
exist" .../>
  <mc:messages mc:msgno="002" mc:msgtext="Airline &amp;1 does not exist" .../>
  ...
</mc:messageClass>
```

關鍵屬性 [adtcore:masterLanguage="EN"](#)——Message Class 有一個「主語言」(原始語言) ，這是後面「怎麼維護多語系」這一段的伏筆：Eclipse ADT 的物件編輯器只顯示、只能編輯這個主語言的文字，這點官方文件後面會直接證實。

⚠ 這一課的 Message Class 物件無法自動建立，需要你在 Eclipse 手動建

比照這門課一貫的查證習慣，先實際嘗試用 MCP 工具自動建立一個測試用 Message Class ([ZFE15_MSG](#)) ，結果跟 [fe12](#) 期末整合已經記錄過的限制一樣：

```
ABAP Object Creation Failed
Object Type: MSAG/N
Error: CREATION_FAILED
```

Message: An error occurred when deserializing in the simple transformation program
ST_ADT_MESSAGE_CLASS

換了套件、也試過帶明確的傳輸請求參數，結果一樣——這是工具本身的限制（跟 fe12 記錄的「`create_object_programmatically` 建 DDLS/PROG/CLAS 全部失敗，同一類 `deserializing error`」同一個模式），不是這個環境不支援 Message Class。沿用課程既有的分工：**Message Class** 物件由你在 **Eclipse** 建立，我負責設計內容跟事後驗證。

Eclipse 建立步驟（請你操作）

1. **Ctrl+Shift+A**（Open ABAP Development Object），或對套件 **ZRAPCLOUD** 右鍵 → **New** → **Other ABAP Repository Object** → 搜尋 **Message Class**
2. Name 填 **ZFE15_MSG**，Description 填 **FE15 Message Class Test**（或任意描述），Package 選 **ZRAPCLOUD**，按 Next / Finish（非 **\$TMP** 套件會跳出 Transport Request 選擇畫面，選一個既有的或新建一個）
3. 建立完成後，編輯器是一個**訊息清單表格**（不是自由文字檔，這是 Message Class 特有的編輯體驗，跟一般 ABAP 原始碼編輯器不同）——新增一筆訊息：
 - **Number**：**001**
 - **Short Text**：**Title and Content must not be identical (Note &1)**

（&1 是佔位符，對應底下 RAP Handler 傳進來的 **v1** 變數，這是 T100 訊息的標準寫法，跟 fe14 用字串模板 `{ ls_note-note_id }` 直接嵌值是不同的機制）

1. 存檔——⚠ **實測確認：Message Class 的訊息文字存檔即生效，不需要（也沒有）Ctrl+F3 啟用這個動作**，跟 DDLS / BDEF / CLAS 這類需要明確 Activate 的 Workbench 物件不同。這跟傳統 **SE91** 的行為一致——T100 訊息文字從以前就沒有「Inactive / Active 版本」這個概念，存檔當下就是最終生效的內容，這也是它跟其他 ABAP Repository 物件最大的行為差異之一，值得記住

在 RAP Validation 裡引用 Message Class：**`new_message()`**，不是 **`new_message_with_text()`**

查證官方 RAP Development Guide（「Implementing the MODIFY」章節）拿到一段真實的官方範例程式碼（不是我推論或猜測的）：

```
ls_travel_reported-%msg = new_message( id          = <fs_message>-symsg-msgid
                                         number     = <fs_message>-symsg-msgno
                                         severity    = if_abap_behv_message=>severity-
error
                                         v1          = <fs_message>-symsg-msgv1
                                         v2          = <fs_message>-symsg-msgv2
                                         v3          = <fs_message>-symsg-msgv3
                                         v4          = <fs_message>-symsg-msgv4 ).
```

`new_message()`（不帶 **`_with_text`**）才是對應 **T100 Message Class** 的方法——跟 fe14 用的 **`new_message_with_text( severity = ... text = ... )`** 是同一個介面（**`IF_ABAP_BEHV_MESSAGE`**）繼承來的兩個不同方法，一個接「Message Class 名稱 + 訊息編號 + 變數」，一個接「純文字」。參數對照：

參數	對應
<code>id</code>	Message Class 名稱 (T100 的 <code>MSGID</code>)，這裡是 <code>'ZFE15_MSG'</code>
<code>number</code>	訊息編號 (T100 的 <code>MSGNO</code>)，這裡是 <code>'001'</code>
<code>severity</code>	跟 <code>new_message_with_text</code> 完全一樣， <code>if_abap_behv_message=>severity-error</code>
<code>v1~v4</code>	對應訊息文字裡的 <code>&1~&4</code> 佔位符

要改的地方在 **Eclipse** 的 `ZBP_I_RC05_NOTE` (Behavior Pool 類別，`Ctrl+Shift+A` 搜尋這個物件名稱打開——不是去找本機快照檔名 `zbp_i_rc05_note.clas.abap`，道理跟 `fe14` 已經澄清過的一樣，那只是這個 `git repo` 存快照用的檔名，Eclipse ADT 認的是物件名稱) → 打開後切到 **Local Types** 分頁 (跟 Global 定義 / 實作是分開的區塊，`fe14` 第一次補內容的地方) → 找到 `lhc_note` 類別裡 `METHOD validateTitleContent` 這個方法本體，把 `%msg` 那一段換成：

```
%msg = new_message( id      = 'ZFE15_MSG'
                    number   = '001'
                    severity = if_abap_behv_message=>severity-error
                    v1       = ls_note-note_id )
```

其餘程式碼 (`READ ENTITIES`、`LOOP`、`IF`、`%element-title / %element-content` 標記) 完全不用改，只換 `%msg` 這一行的來源。這樣訊息文字就不再寫死在 ABAP 程式碼裡，而是引用 `ZFE15_MSG 001` 這則訊息——這正是「可重複使用、可翻譯」的關鍵：之後如果十個不同的 `Validation` 都要顯示「兩個欄位不可相同」這種訊息，可以共用同一則 T100 訊息，不用像 `new_message_with_text` 那樣在每個地方各自寫一次文字。

多語系怎麼維護：Maintain Translations App (取代 `SE63 / SOTR_EDIT`)

查證官方文件 (BTP ABAP Environment 「SAP Fiori Applications in the ABAP Environment」章節) 明確寫著：

To handle texts from the ABAP development objects, you use the **Maintain Translations App**. This application provides a user-friendly interface for managing translations within the ABAP environment.

這是一個 **Fiori Launchpad** 上的 **App**，不是 **Eclipse** 裡的功能——官方文件同時明確提醒一個容易誤解的地方：

The ADT editor will only display the original [language] texts.

也就是說：你沒辦法在 **Eclipse** 直接切換語言、把 `ZFE15_MSG` 的訊息文字改成中文或德文再存一次——Eclipse 的 Message Class 編輯器只認得、只能編輯建立時的主語言 (這裡是 `EN`)，翻譯一定要透過 **Maintain Translations App** 這條路。

完整流程 (查證官方文件逐步核對，並已由使用者實機走過一遍確認)

1. Fiori Launchpad 搜尋並開啟 **Maintain Translations** 這個 App——網址規律是部署網域把 `abap` 換成 `abap-web`，加上 `/sap/bc/ui2/flp` (呼應 `fe11` 已記錄的規律)，登入後搜尋框直接打 **Maintain Translations**
2. 建立新 **Project** (點畫面右上角 +)，填 ID (例如 `ZFE15_TRANS`) + Description——☒ 實測踩到一個畫面行為：在「建立專案」這個對話框裡，**Text Sources** 的 **Add** 按鈕是反灰、按不下去的，一定要先

把專案存檔建立好，回到專案詳細畫面才能加 **Text Source**，這是兩個分開的步驟，不是一個精靈流程走到底

- 專案建好後，切到 **Text Sources** 分頁，這時候 **Add** 才能點——選 **Type = Message classes**，篩選 **Name = ZFE15_MSG**，加入專案（官方文件列出的支援類型清單裡，**Message classes** 是明確支援的一種）
- 切到 **Translations** 分頁，點 **Create**，設定一組 **Source Language**（**English United States / EN**）→ **Target Language**（例如 **Chinese traditional (Taiwan) / zh-Hant**）
- 勾選（單選）剛剛那一列 → **Download** → 選 **All Texts** → 下載一份 **XLIFF** 檔（**.xlf**，本質是 XML，用 VS Code 打開）
- ✅ 實測發現：第一次翻譯的 **XLIFF** 裡完全沒有 **<target>** 元素，只有 **<source>**（原文）跟 **<note>**（說明）——因為這是這則訊息第一次被翻譯，沒有任何舊翻譯內容可以預先產生。要自己手動新增一個 **<target>** 元素，放在 **</source>** 之後、**<note>** 之前：

```
`xml <trans-unit id="MESSAGE_CLASS:ZFE15_MSG:SHORT_TEXT:001" maxwidth="73" size-unit="char"> <source>Title and Content must not be identical (Note &1)</source> <target>標題與內容不可相同 (單據 &1)</target> <note>Short description for message 001 of message class ZFE15_MSG.</note> </trans-unit> ` &1 這個轉義字元直接照抄 <source> 裡的寫法，不要自己手動打 &1`（會破壞 XML 語法），存檔維持 UTF-8 編碼
```

- 回到 Maintain Translations App，確認同一列還是選取狀態 → **Upload** → 選剛剛改好的 XLIFF 檔案上傳——上傳成功後 **Last Uploaded At / Last Uploaded By** 欄位會出現時間戳記與使用者 ID
- 上傳完還不算生效，同一列 → **Publish** → 選一個 Transport Request 確認——Publish 成功後 **Last Published At / Last Published By** 欄位也會出現值
- Publish 完成後，翻譯後的文字才算真正寫進系統

⚠️ 官方文件提到的限制，值得先確認再動手

Text sources that can be translated by a translation project need to reside in the **same software component** as the translation project.

這代表 **ZFE15_MSG** 所在的套件（**ZRAPCLOUD**）跟你在 Maintain Translations 建立的 Translation Project，兩者要落在同一個 Software Component——這個共用 Trial 系統目前所有自訂套件通常都在同一個 Software Component 下，理論上不會卡關，但如果 Add Text Source 那一步找不到 **ZFE15_MSG**，第一件事就是回頭檢查這個限制。

怎麼證明畫面上的訊息真的來自 Message Class，不是巧合文字一樣

fe14 / fe15 這兩課刻意把訊息文字設計成一樣（**Title and Content must not be identical (Note ...)**）——單純看畫面，完全分不出這則訊息是 **new_message_with_text** 寫死的、還是 **new_message** 從 **ZFE15_MSG** 001 讀出來的。這是故意的：逼你想清楚「怎麼證明程式碼真的照你以為的方式在跑」，不能只看結果長得像不像。

最直接、最有說服力的驗證方法：只改 **Message Class** 的訊息文字，完全不碰 **ABAP** 程式碼，看畫面會不會跟著變：

- 回 Eclipse 打開 **ZFE15_MSG**（**Ctrl+Shift+A** 搜尋這個物件名稱），把 001 這則訊息的 Short Text 改一眼就看得出差異的版本，例如 **>>> TEST FROM MESSAGE CLASS <<< Title &1 vs Content**

2. 存檔即生效 (前面已經確認過，訊息文字不用 **Ctrl+F3** 啟用) ——注意這一步完全沒有碰 **ZBP_I_RC05_NOTE**，**validateTitleContent** 方法一個字都沒改
3. 回 VS Code App，重新整理，再測一次 Title = Content 的情境
4. 如果畫面上的訊息文字變成你剛改的那個版本，就證實了這則訊息是執行期才去讀 **ZFE15_MSG** 001 的內容，不是編譯時期就固定死在程式碼裡的字串——這是「可重複使用、可翻譯」這個機制成立的關鍵前提，值得親手驗證一次，不要只憑語法看起來合理就假設它真的這樣運作

驗證完別忘了把訊息文字改回正式版本 (**Title and Content must not be identical (Note &1)**)，不然後面的多語系翻譯練習會對到錯誤的來源文字。

次要驗證：瀏覽器開發者工具 **Network** 分頁

呼應 fe14 已經教過的技巧——打開 **Network** 分頁，找到觸發驗證失敗的那個 **PATCH / Activate** 請求，看它的回應內容 (**Response Body**)。⚠ 這裡要老實說：我沒辦法事先確認這個 **OData V4** 錯誤回應的 **JSON** 格式裡，會不會直接曝露 **ZFE15_MSG / 001** 這兩個技術值——如果你翻到了，那是最直接的技術證據；如果完全沒看到 (很可能整個回應只有渲染好的訊息文字，沒有技術 ID)，也不代表判斷錯了，只代表這個框架版本沒有把 **Message Class** 的技術資訊往外曝露給前端。上面「改文字看畫面會不會變」才是保證有效、不用看框架臉色的驗證方式，**Network** 分頁只是順手可以多看一眼的輔助資訊。

驗證方式

✅ 前三項已由使用者實機驗證確認 (2026-08-24)：

1. **ZFE15_MSG** (**Message Class**) 在 **Eclipse** 建立成功——確認訊息文字存檔即生效，不需要 **Ctrl+F3** 啟用 (呼應上面 **Eclipse** 建立步驟第 4 點的更正)
2. **ZI_RC05_NOTE / ZBP_I_RC05_NOTE** 的 **validateTitleContent** 改用 **new_message(id = 'ZFE15_MSG' number = '001' ...)** 後啟用成功
3. 重新測 fe14 Part B 的情境 (Title = Content)，確認訊息正確跳出來；進一步做了「改 **Message Class** 訊息文字、不碰 **ABAP** 程式碼」的關鍵驗證 (見上面「怎麼證明畫面上的訊息真的來自 **Message Class**」段落)——把 001 訊息改成 **>>> TEST FROM MESSAGE CLASS <<< Title &1 vs Content** 後，畫面截圖確認訊息文字真的變成這個測試版本，證實 **new_message** 是執行期讀取 **Message Class** 內容，不是編譯時期固定死的字串。測完已改回正式文字 **Title and Content must not be identical (Note &1)**

✅ 項目 4 / 5 已由使用者實機驗證確認 (2026-08-25)：

1. **Fiori Launchpad** 上順利找到並開啟 **Maintain Translations** App——這個共用帳號在這個 App 上沒有遇到 fe11 記錄過的權限邊界問題，可以正常使用
2. 完整走完 **Create Project (ZFE15_TRANS)** → 加入 **Text Source (ZFE15_MSG)** → 設定 **Source/Target Language (EN→zh-Hant)** → 下載 **XLIFF** → 手動新增 **<target>** 節點 → 上傳 → **Publish** 全流程，畫面截圖確認 **Last Uploaded At / Last Published At** 都正確出現時間戳記與使用者 ID

✅ 項目 6 已由使用者實機驗證確認，這一課全部六項驗證正式完成 (2026-08-25)：

1. 在 **VS Code App** 的網址加上 **sap-language=zh-TW** 這個標準 **SAPUI5** 啟動參數 (**http://localhost:8080/test/flp.html?sap-language=zh-TW&sap-ui-xx-viewCache=false#app-preview&...**)，重新整理後重測 Title = Content 情境，訊息文字正確顯示繁體中文翻譯：「標題與內容不可相同 (單據 testdfs)」，**&1** 佔位符正確代入 **note_id**。額外發現：**sap-language** 這個參數不只讓我們自訂的 **Message Class** 訊息變成中文，連 **Fiori Elements** 框架本身

的標準 UI 文字也一起變成中文 (「Message Details」→「訊息明細」、「Draft updated」→「已更新草稿」、「Create」→「建立」、「Discard Draft」→「捨棄草稿」)，證實這個參數是全域生效的標準 i18n 機制，不是只影響後端 OData 請求

學習目標

- 知道 Message Class (**MSAG**) 在 ABAP Cloud 環境完全沒有 T-code，只能在 Eclipse ADT 建立與編輯訊息清單 (表格式編輯器，不是自由文字)
- 能講出 **new_message()** (T100 Message Class) 跟 **new_message_with_text()** (臨時文字) 的語法差異與取舍：前者可重複使用、可翻譯，但要多建一個物件；後者零額外物件，但文字寫死、不能共用也不能翻譯
- 知道 Eclipse ADT 的物件編輯器只能顯示 / 編輯 Message Class 的主語言 (**masterLanguage**) 文字，多語系翻譯完全是另一條路
- 能講出 **Maintain Translations App** 完整流程：Create Project → Add Text Source (Message classes 是官方支援的來源類型) → 設定 Source/Target Language → 下載 XLIFF → 翻譯 → 上傳 → Publish (含選 Transport)
- 知道「Text Source 要跟 Translation Project 同一個 Software Component」這個容易卡關的前提條件
- 知道 **sap-language** 是標準 SAPUI5 啟動參數，加在 App 網址上會讓整個框架 (含標準按鈕文字) 跟自訂的 Message Class 訊息一起切換語言，不是只影響其中一種

物件清單

物件	型別	說明
ZFE15_MSG	MSAG (Message Class)	這一課新建，套件 ZRAPCLOUD ，需使用者在 Eclipse 建立 (自動建立工具失敗)
ZI_RC05_NOTE	BDEF	validateTitleContent 的 %msg 來源從 new_message_with_text 改成 new_message
ZBP_I_RC05_NOTE	CLAS (Local Types)	同上， lhc_note 類別內容更新

前端沒有新增或修改任何檔案，沿用 **fe01_connection_test**。

動手練習

- 想一想：如果 **ZFE15_MSG** 001 這則訊息之後要被另一個完全不相干的 RAP BO (例如 **fe08** 的 **ZI_RC01_TASK**) 拿去用，語法上要怎麼寫？ (提示：**new_message** 的 **id / number** 參數本身就是通用的，跟呼叫端是哪個 BDEF 完全無關)
- 承上，如果兩個不同的 BO 共用同一則訊息，但其中一個想要 Warning、另一個想要 Error，這個差異要在哪裡調整？ (提示：**severity** 是呼叫端自己傳的參數，不是訊息本身固定的屬性)
- 完成 Maintain Translations 流程後，想一想：這個「Create Project → Add Source → Download XLIFF → 上傳 → Publish」的流程，如果之後 **ZFE15_MSG** 又加了第 002 則訊息，要重新走一次完整流程嗎？ (提示：回顧「Generate and Download the XLIFF File」文件裡「All Texts」跟「Changed Texts」這兩個選項的差異)

驗證方式

見上面 Lecture 段落的「驗證方式」，六項全部已由使用者實機操作確認完成（2026-08-25）。

思考題

1. 這一課證實了 Eclipse ADT 編輯器「只能顯示/編輯主語言」——如果你不小心把 Message Class 的 `masterLanguage` 設錯（例如專案團隊主要用中文開發，但建立時 Master Language 選了英文），之後想「換主語言」，你覺得系統會不會提供這個選項？這對「新建 Message Class 時要先想清楚 Master Language」這件事有什麼啟發？
2. `new_message` 的 `v1~v4` 只有四個變數位置——如果一則訊息需要超過 4 個動態值（例如同時要顯示 Note ID、Title、Content、修改時間），語法上要怎麼處理？（提示：想一想訊息文字本身的長度限制，跟「是不是應該把某些資訊移到訊息以外的地方顯示」這個設計問題）
3. 對照 fe14 學到的「`field(mandatory)` 只是 UI 提示、沒有執行期檢查」——這一課的 Message Class 訊息文字，如果你透過 Maintain Translations 上傳了一個目標語言版本、但還沒有 **Publish**，這時候如果剛好有人用該語言登入系統存取這個訊息，看到的會是什麼？（提示：回顧上傳跟發佈是兩個獨立步驟這件事）

答案

見 ZFE15_MSG (Message Class · 001 訊息)、ZI_RC05_NOTE / ZBP_I_RC05_NOTE (`validateTitleContent` 改用 `new_message`)、ZFE15_TRANS (Maintain Translations 專案)。這一課全部六項驗證已端對端實機完成（2026-08-25）：Message Class 建立與執行期讀取、`new_message` 語法、Maintain Translations 完整 XLIFF 流程（含手動補 `<target>` 元素）、以及最後用 `sap-language=zh-Tw` 網址參數確認翻譯後文字真的在畫面上生效，連框架標準文字都一併切換語言——延伸篇 fe13 ~ fe15 至此全部完整驗證，正式結案。